

A.Tejaswini - 192471043

TOPIC 3: DIVIDE AND CONQUER

#1 Find Minimum and Maximum

Program

```
def find_min_max(arr):

    minimum = arr[0]
    maximum = arr[0]

    for i in arr:

        if i < minimum:
            minimum = i

        if i > maximum:
            maximum = i

    return minimum, maximum

arr = [5,7,3,4,9,12,6,2]

mn, mx = find_min_max(arr)

print("Min =", mn)
print("Max =", mx)
```

Output

Min = 2
Max = 12

#2 Minimum and Maximum (Sorted Array)

Program

```
def min_max(arr):  
  
    print("Min =", arr[0])  
    print("Max =", arr[-1])  
  
arr = [2,4,6,8,10,12,14,18]  
  
min_max(arr)
```

Output

```
Min = 2  
Max = 18
```

#3 Merge Sort

Program

```
def merge_sort(arr):  
  
    if len(arr) <= 1:  
        return arr  
  
    mid = len(arr)//2  
  
    left = merge_sort(arr[:mid])  
  
    right = merge_sort(arr[mid:])  
  
    result = []  
  
    i = j = 0  
  
    while i < len(left) and j < len(right):  
  
        if left[i] < right[j]:
```

```

        result.append(left[i])

        i += 1

    else:

        result.append(right[j])

        j += 1

    result += left[i:]
    result += right[j:]

    return result

arr = [31,23,35,27,11,21,15,28]

print(merge_sort(arr))

```

Output

```
[11, 15, 21, 23, 27, 28, 31, 35]
```

#4 Merge Sort with Comparison Count

Program

```

count = 0

def merge_sort(arr):

    global count

    if len(arr) <= 1:
        return arr

    mid = len(arr)//2

```

```
left = merge_sort(arr[:mid])

right = merge_sort(arr[mid:])

result = []

i = j = 0

while i < len(left) and j < len(right):

    count += 1

    if left[i] < right[j]:

        result.append(left[i])

        i += 1

    else:

        result.append(right[j])

        j += 1

result += left[i:]
result += right[j:]

return result
```

```
arr = [12,4,78,23,45,67,89,1]
```

```
print(merge_sort(arr))
print("Comparisons =", count)
```

Output

```
[1, 4, 12, 23, 45, 67, 78, 89]
```

```
Comparisons = 14
```

#5 Quick Sort (First Pivot)

Program

```
def quick_sort(arr):  
  
    if len(arr) <= 1:  
        return arr  
  
    pivot = arr[0]  
  
    left = [x for x in arr[1:] if x <= pivot]  
  
    right = [x for x in arr[1:] if x > pivot]  
  
    return quick_sort(left) + [pivot] + quick_sort(right)
```

```
arr = [10,16,8,12,15,6,3,9,5]
```

```
print(quick_sort(arr))
```

Output

```
[3, 5, 6, 8, 9, 10, 12, 15, 16]
```

#6 Quick Sort (Middle Pivot)

Program

```
def quick_sort(arr):  
  
    if len(arr) <= 1:  
        return arr  
  
    pivot = arr[len(arr)//2]  
  
    left = [x for x in arr if x < pivot]  
  
    middle = [x for x in arr if x == pivot]
```

```
right = [x for x in arr if x > pivot]
```

```
return quick_sort(left) + middle + quick_sort(right)
```

```
arr = [19,72,35,46,58,91,22,31]
```

```
print(quick_sort(arr))
```

Output

```
[19, 22, 31, 35, 46, 58, 72, 91]
```

#7 Binary Search with Comparison Count

Program

```
def binary_search(arr, key):
```

```
    low = 0
```

```
    high = len(arr)-1
```

```
    count = 0
```

```
    while low <= high:
```

```
        count += 1
```

```
        mid = (low+high)//2
```

```
        if arr[mid] == key:
```

```
            return mid+1, count
```

```
        elif arr[mid] < key:
```

```
            low = mid+1
```

```
else:
```

```
    high = mid-1
```

```
return -1, count
```

```
arr = [5,10,15,20,25,30,35,40,45]
```

```
pos, c = binary_search(arr,20)
```

```
print("Position =", pos)
```

```
print("Comparisons =", c)
```

Output

Position = 4

Comparisons = 2

#8 Binary Search

Program

```
def binary_search(arr, key):
```

```
    low = 0
```

```
    high = len(arr)-1
```

```
    while low <= high:
```

```
        mid = (low+high)//2
```

```
        if arr[mid] == key:
```

```
            return mid+1
```

```
        elif arr[mid] < key:
```

```
        low = mid+1

    else:

        high = mid-1

    return -1

arr = [3,9,14,19,25,31,42,47,53]

print(binary_search(arr,31))
```

Output

6

#9 K Closest Points

Program

```
def k_closest(points, k):

    points.sort(
        key=lambda p:
            p[0]**2 + p[1]**2
    )

    return points[:k]

points = [
    [1,3],
    [-2,2],
    [5,8],
    [0,1]
]
```

```
print(k_closest(points,2))
```

Output

```
[[ -2,  2], [ 0,  1]]
```

#10 Four Sum Count

Program

```
def four_sum(A,B,C,D):
```

```
    count = 0
```

```
    for a in A:
```

```
        for b in B:
```

```
            for c in C:
```

```
                for d in D:
```

```
                    if a+b+c+d == 0:
```

```
                        count += 1
```

```
    return count
```

```
A=[1,2]
```

```
B=[-2,-1]
```

```
C=[-1,2]
```

```
D=[0,2]
```

```
print(
```

```
four_sum(  
A,B,C,D  
)  
)
```

Output

2

#11 Median of Medians

Program

```
def median_of_medians(arr, k):
```

```
    arr.sort()
```

```
    return arr[k-1]
```

```
arr = [12,3,5,7,19]
```

```
k = 2
```

```
print(  
median_of_medians(  
arr,  
k  
)  
)
```

Output

5

#12 K-th Smallest Element

Program

```
def kth_smallest(arr, k):
```

```
    arr.sort()
```

```
return arr[k-1]
```

```
arr = [  
23,  
17,  
31,  
44,  
55,  
21,  
20,  
18,  
19,  
27  
]
```

```
k = 5
```

```
print(  
kth_smallest(  
arr,  
k  
)  
)
```

Output

21

#13 Meet in the Middle (Closest Sum)

Program

```
from itertools import combinations
```

```
arr = [  
45,
```

```
34,  
4,  
12,  
5,  
2  
]
```

```
target = 42
```

```
best = 0
```

```
for r in range(  
len(arr)+1  
):
```

```
    for c in combinations(  
        arr,  
        r  
    ):
```

```
        s = sum(c)
```

```
        if abs(  
            target-s  
        ) < abs(  
            target-best  
        ):
```

```
            best = s
```

```
print(best)
```

Output

41

#14 Meet in the Middle (Exact Sum)

Program

```
from itertools import combinations
```

```
arr = [  
    1,  
    3,  
    9,  
    2,  
    7,  
    12  
]
```

```
target = 15
```

```
found = False
```

```
for r in range(  
    len(arr)+1  
):
```

```
    for c in combinations(  
        arr,  
        r  
    ):
```

```
        if sum(c) == target:
```

```
            found = True
```

```
print(found)
```

Output

```
True
```

#15 Matrix Multiplication

Program

```
A = [  
[1,7],  
[3,5]  
]
```

```
B = [  
[6,8],  
[4,2]  
]
```

```
C = [  
[0,0],  
[0,0]  
]
```

```
for i in range(2):
```

```
    for j in range(2):
```

```
        for k in range(2):
```

```
            C[i][j] += (  
                A[i][k]  
                *  
                B[k][j]  
            )
```

```
for row in C:
```

```
    print(row)
```

Output

[34, 22]

[38, 34]

#16 Karatsuba Multiplication

Program

```
def karatsuba(x, y):
```

```
    if x < 10 or y < 10:
```

```
        return x*y
```

```
    n = max(
        len(str(x)),
        len(str(y))
    )
```

```
    m = n//2
```

```
    high1 = x//10**m
```

```
    low1 = x%10**m
```

```
    high2 = y//10**m
```

```
    low2 = y%10**m
```

```
    z0 = karatsuba(
        low1,
        low2
    )
```

```
    z1 = karatsuba(
        low1+high1,
        low2+high2
    )
```

```

z2 = karatsuba(
    high1,
    high2
)

return (

    z2*10**(2*m)

    +

    (z1-z2-z0)
    *
    10**m

    +

    z0

)

print(
    karatsuba(
    1234,
    5678
    )
    )

```

Output

7006652